JPEG-LS ENCODER IP CORE · V1.4

OpenJLS

Lossless JPEG-LS encoder for FPGAs.
One pixel per clock, no external memory.

340 MPx/s	1.6:1	8–16 bit	11639/11639
THROUGHPUT, 12-BIT	COMPRESSION	PIXEL DEPTH	BYTE-EXACT IMAGES

01

Overview

OpenJLS is a lossless JPEG-LS encoder IP core for FPGAs. It takes one grayscale pixel per clock over a ready/valid stream and emits a standard JPEG-LS bitstream, buffering a single image row on-chip. It is portable VHDL, verified byte-exact against the CharLS reference in simulation and on silicon.

Source and verification suite: `https://github.com/isentropic-fpga/OpenJLS`
Demo projects: `https://github.com/isentropic-fpga/OpenJLS-Demos`

Table 1 Key specifications.

Standard	JPEG-LS[1]
Compression	Lossless
Compression ratio	≈1.6:1[2]
Components	Grayscale
Pixel depth	8–16 bit[3]
Image size	Up to ≈64k × 64k px[4]
Throughput	1 pixel / clock
Latency	14 cycles
f_{max}	≈340 MHz[5]
Interface	Ready/valid stream[6]
Memory	On-chip only[7]
Resources	5.4–7.7k LUT, 1.9–2.4k FF[8]
Portability	Vendor-agnostic VHDL

02

Functional description

The core is a fully pipelined encoder with a latency of 14 clock cycles: an input register, six processing stages (gradient computation, context modeling, MED prediction with bias correction, Golomb–Rice / run-length coding) and seven internally registered output stages (bit packer, byte stuffer, framer). The architecture builds on Mert [2]; RAM primitives come from open-logic [4].

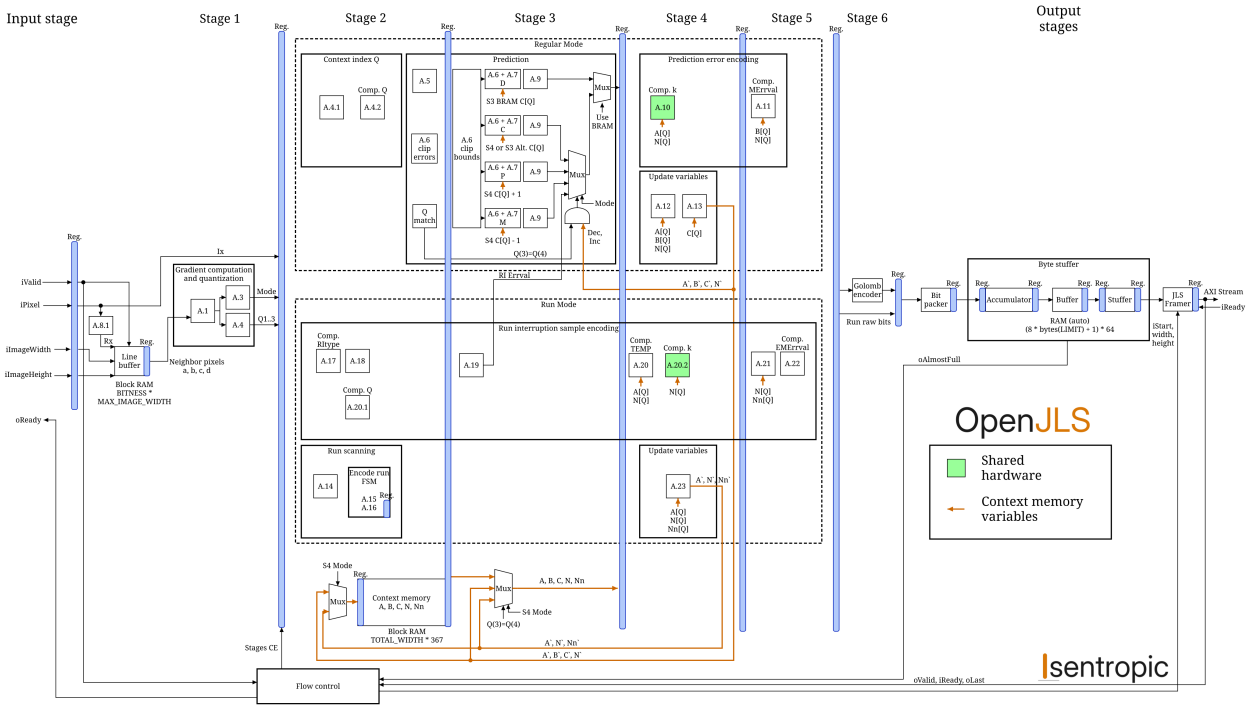
End-of-image is derived internally from the dimensions provided at the ports `iImageWidth` and `iImageHeight`. The core never stalls at end of line or end of image, so images stream back-to-back. It stalls only when downstream backpressure fills the byte-stuffer buffer, or, for back-to-back images of 8 pixels or fewer, while three images are in flight. In principle the byte stuffer's fixed 4-byte-per-clock drain could also stall the input if the code rate exceeded it for long enough, but none of the random, adversarial or worst-case natural images we measured caused a stall.

The output stages assert `oValid` only once a full `OUT_WIDTH` word is ready, so output beats are much rarer than

[1] ISO/IEC 14495-1, ITU-T T.87.
[2] Aggregate over the 240 natural photographs of the test corpus; content-dependent.
[3] Set by the `BITNESS` generic.
[4] Nominally 65535 × 65535 (64k − 1), the largest value that fits the 2-byte dimension fields.
[5] 12-bit, Xilinx `xczu7eg-1`.
[6] AXI4-Stream and Avalon-ST compatible.
[7] One image row; no external RAM.
[8] Across 8–16 bit, plus block RAM that scales with image width and pixel depth.

input pixels.

Figure 1 Encoder pipeline architecture.



A full-resolution version of this diagram is available in the repository.

Flow control drives every stage's clock enable and starts the framer on each image's first pixel.

Context memory holds A, B, C, N and Nn for each context. A token reads it in S2 and receives the data in S3, but the BRAM word is wrong if the context hasn't been used yet in this image, or if a token up to three positions ahead just updated it. Per-context init flags handle the first case, so the memory resets at end of image without rewriting the RAM. In the second, the memory forwards writes from tokens two and three positions ahead, while a token one position ahead is still in S4, so its update comes from there.

Regular-mode prediction runs four parallel chains, because the correct C[Q] comes too late in the cycle to select first. Chain D takes the BRAM word. Chain C takes S4's C[Q] on a match with the token ahead, otherwise the memory's alternative (Alt.) word: the init value or a forwarded write. Chains P and M take S4's C[Q] + 1 and - 1, covering A.13's possible results. The final select picks one chain's result just before Reg3.

03

Generics

Table 2 Configuration generics.

GENERIC	RANGE	DESCRIPTION
BITNESS	8–16	Pixel bit depth.
MAX_IMAGE_WIDTH	4–65535	Largest width supported; sets line-buffer depth.
MAX_IMAGE_HEIGHT	1–65535	Largest height supported.
OUT_WIDTH	48–1024	Output bus width in bits; multiple of 8.

MAX_IMAGE_WIDTH and MAX_IMAGE_HEIGHT set the *compile-time* maximum and size the line buffer and dimension counters. Each image's actual size is set at *run time* through iImageWidth and iImageHeight (Table 3), up to that

maximum.

04

Ports

All ports are synchronous to the rising edge of `iClk` — the core is a single clock domain. The streaming ports use a plain ready/valid handshake; in Table 3 the **AXIS** column gives the one-to-one AXI4-Stream signal mapping for that ecosystem (Avalon-ST maps the same way at `readyLatency` = 0).

Table 3 Port list and AXI4-Stream mapping.

PORT	DIR	WIDTH	AXIS	ROLE
`iClk`	in	1	—	Clock.
`iRst`	in	1	—	Synchronous reset, active high; latches dimensions.
`iImageWidth`	in	16	—	Image width, pixels (configuration).
`iImageHeight`	in	16	—	Image height, pixels (configuration).
`iValid`	in	1	`TVALID`	Pixel valid.
`iPixel`	in	`BITNESS`	`TDATA`	Pixel data.
`oReady`	out	1	`TREADY`	Input ready.
`oData`	out	`OUT_WIDTH`	`TDATA`	Bitstream beat.
`oValid`	out	1	`TVALID`	Output valid.
`oKeep`	out	`OUT_WIDTH/8`	`TKEEP`	Valid-byte mask on final beat.
`oLast`	out	1	`TLAST`	End of image.
`iReady`	in	1	`TREADY`	Downstream ready.

IMPORTANT:

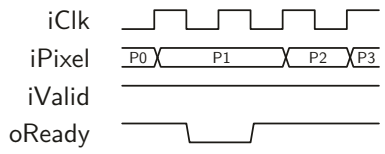
- `iImageWidth` and `iImageHeight` are only sampled while `iRst` is asserted (`'1'`).
- Out-of-range values (width outside 4–`MAX_IMAGE_WIDTH`, height outside 1–`MAX_IMAGE_HEIGHT`) fall back to the maximum.
- If you don't intend to change the image dimensions at run-time, wire both inputs to `0` so that the maximum values are used.
- Both ports are a fixed 16 bits regardless of the `MAX_IMAGE_*` generics, which keeps them usable in Vivado block designs (their port-width evaluator handles only literal arithmetic).

05

Timing and protocol

Pixel input. Pixels are streamed in scan order — the first row left to right, then the second row, and so on — each an unsigned value on `iPixel`. A pixel transfers on every cycle where `iValid` and `oReady` are both high. Outside reset, the core holds `oReady` high and drops it only when the byte-stuffer buffer fills under downstream back-pressure. Figure 2 shows a transfer with a brief downstream stall.

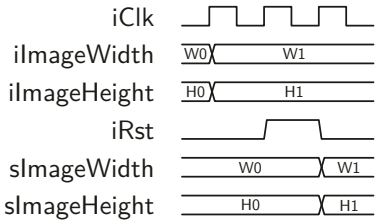
Figure 2 Pixel-input handshake.



No pixel is dropped while oReady is low.

Reset and configuration. iRst is level-sensitive (hold it high for at least one clock) and clears the pipeline. The image dimensions are captured only while iRst is high: present iImageWidth and iImageHeight, hold them stable across the reset pulse, and they latch on the rising edge (Figure 3). A reset is therefore needed only to capture the dimensions — before the first image and whenever the image size changes; with the size unchanged, images encode back-to-back.

Figure 3 Reset and dimension capture.



sImageWidth and sImageHeight are the internal registers that hold the dimensions; they load on the rising edge while iRst is high.

Output. Beats appear on oData/oValid subject to iReady; oLast marks the final beat (EOI) and oKeep flags its valid bytes. The bitstream is byte-serial and MSB-first: the earliest byte occupies the most-significant byte of oData.

06

Integration example

```
u_enc : entity work.openjls_top
  generic map (
    BITNESS => 8,
    MAX_IMAGE_WIDTH => 4096,
    MAX_IMAGE_HEIGHT => 4096,
    OUT_WIDTH => 64 )
  port map (
    iClk => clk,
    iRst => rst,
    iImageWidth => img_w, -- latched on reset
    iImageHeight => img_h, -- latched on reset
    iValid => s_valid, -- pixel stream (AXIS slave)
    iPixel => s_data,
    oReady => s_ready,
    oData => m_tdata, -- bitstream (AXIS master)
    oValid => m_tvalid,
    oKeep => m_tkeep,
    oLast => m_tlast,
    iReady => m_tready );
```

It can also be added to a block design as an RTL module.

Xilinx IP cores

For the Vivado flow the core ships the following pre-packaged IPs:

IP CATALOG NAME	INTERFACES
OpenJLS Encoder (Native)	Plain ready/valid
OpenJLS Encoder (AXI4-Stream)	AXI-Stream for the data, native pins for control
OpenJLS Encoder (AXI4-Stream + AXI4-Lite)	AXI-Stream for the data, AXI-Lite for control

Add `Sources/Xilinx/ip_repo/` to the project's IP repositories and the three cores appear in the IP Catalog, ready to drop onto a block design.

AXI4-Lite register map

The AXI4-Lite variant replaces the native control pins with a register bank (32-bit registers, word-aligned offsets):

OFFSET	NAME	ACCESS	CONTENTS
`0x00`	ID	RO	ASCII "OJLS" (`0x4F4A4C53`)
`0x04`	VERSION	RO	`0x00MMmmpp` (major/minor/patch)
`0x08`	CAPS	RO	[7:0] `BITNESS`, [15:8] output bytes per beat (`OUT_WIDTH`/8)
`0x0C`	MAXDIM	RO	[15:0] `MAX_IMAGE_WIDTH`, [31:16] `MAX_IMAGE_HEIGHT`
`0x10`	WIDTH	RW	[15:0] image width (clamped on write)
`0x14`	HEIGHT	RW	[15:0] image height (clamped on write)
`0x18`	CTRL	WO	[0] APPLY — self-clearing, pulses the core reset
`0x1C`	STATUS	RO	[0] BUSY (reset pulse active), [1] pixel-stream TREADY mirror

The core samples the dimensions only while its reset is high, so reconfiguration is: write WIDTH/HEIGHT, set CTRL.APPLY. APPLY pulses the core reset for one clock while the registers are held stable; the AXI-Lite endpoint itself stays on `aresetn` only, so the bus never disappears mid-reconfigure. While the pulse is active STATUS.BUSY reads 1 and the pixel stream's TREADY is held low, so a stream started too early stalls instead of losing pixels — WIDTH/HEIGHT/CTRL writes while BUSY are dropped. **APPLY aborts any image in flight — only apply between images.** Back-to-back images of unchanged dimensions need no APPLY.

WIDTH/HEIGHT writes are merged per WSTRB and clamped to the core's rule (out-of-range values become the MAX generic), so a readback always returns the value the core will actually use. Writes to RO or unmapped offsets are acknowledged (OKAY) and dropped; unmapped reads return zero.

For driver code there is a C header in the repository, ojls_regs.h.

07

Resources and performance

Characterized on Xilinx Zynq UltraScale+ `xczu7eg-fbvb900-1-e`, Vivado 2025.2, with `OUT_WIDTH = 64`, RTL-only with no floorplanning. f_{max} is the true maximum, found by over-constraining the clock until timing fails. At one pixel per clock, ≈340 MHz is ≈340 Mpixel/s.

Every configuration is implemented with four strategies: the Vivado default, EPRPO (`Performance_ExplorePostRoutePhysOpt`), NetDelay (`Performance_NetDelay_high`) and Congestion (`Congestion_SpreadLogic_high`). Frequencies are in MHz; bold marks the best strategy per row. Resources are for the default strategy and near-identical across strategies.

Image width sweep

`BITNESS = 12`, `MAX_IMAGE_WIDTH` from 4096 to 65535. The design is congestion-bound, so the winning strategy shifts with image size and with netlist changes; the best-of band (≈335–359 MHz) is the headline figure. Strategies differ by 10–32 MHz per size, and a default run lands up to ≈27 MHz below it. Logic is essentially constant with image size; only the line-buffer BRAM scales with width.

Figure 4 Maximum frequency vs maximum image width.

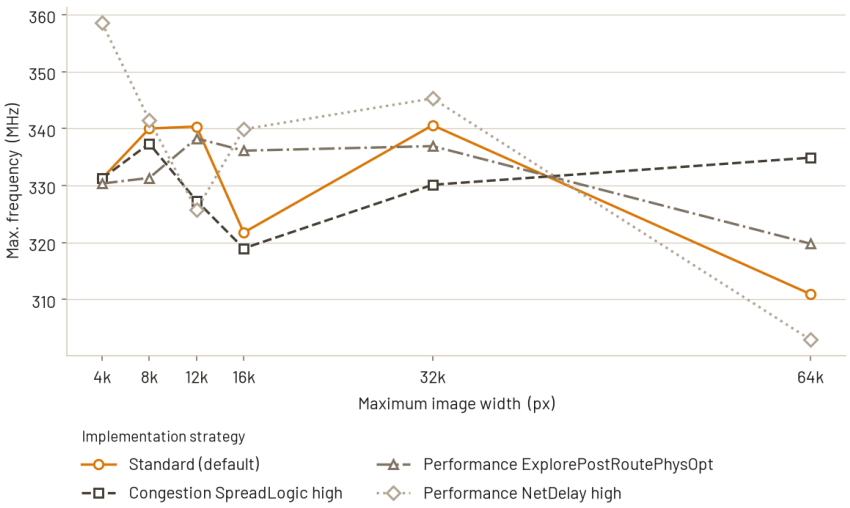


Table 4 Maximum frequency by maximum image width and strategy.

MAX_IMAGE_WIDTH	DEFAULT	EPRPO	NETDELAY	CONGESTION
4096	331.2	330.4	**358.6**	331.2
8192	340.0	331.4	**341.5**	337.4
12288	**340.4**	338.3	325.7	327.3
16384	321.8	336.1	**339.9**	319.0
32768	340.6	336.9	**345.3**	330.1
65535	310.9	319.8	302.9	**334.9**

Figure 5 Resource usage vs maximum image width.

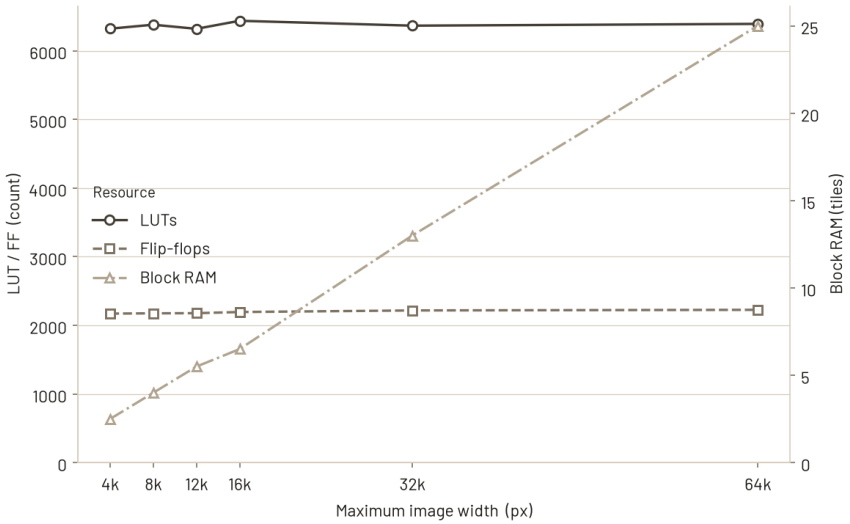


Table 5 Resource usage by maximum image width.

MAX_IMAGE_WIDTH	LUT	FF	BRAM
4096	6322	2172	2.5
8192	6380	2176	4.0
12288	6320	2178	5.5
16384	6437	2195	6.5
32768	6366	2218	13.0
65535	6392	2227	25.0

Pixel depth sweep

`MAX_IMAGE_WIDTH = 12288`, `BITNESS` from 8 to 16. Best-of f_{max} falls from ≈360 MHz at 8 and 10 bits to ≈325 MHz at 14 and 16 bits. The limit is the context update loop and the context RAM read into the prediction-error register, whose carry chains widen with pixel depth; at 8 bits the byte stuffer limits too. Logic and line-buffer BRAM both grow with depth.

Figure 6 Maximum frequency vs pixel bit depth (`MAX_IMAGE_WIDTH = 12288`).

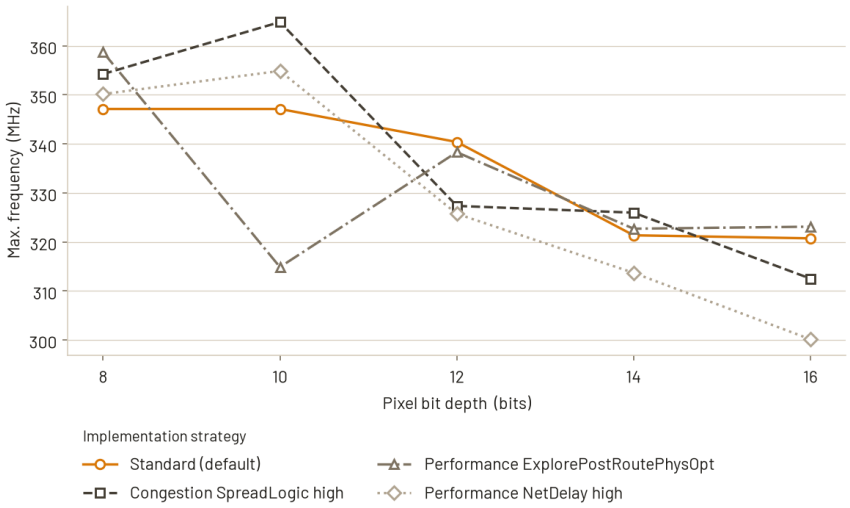


Table 6 Maximum frequency by pixel bit depth and strategy (`MAX_IMAGE_WIDTH = 12288`).

BITNESS	DEFAULT	EPRPO	NETDELAY	CONGESTION
8	347.1	**358.7**	350.1	354.2
10	347.1	314.9	354.9	**364.8**
12	**340.4**	338.3	325.7	327.3
14	321.3	322.7	313.7	**325.9**
16	320.7	**323.1**	300.1	312.5

Figure 7 Resource usage vs pixel bit depth (`MAX_IMAGE_WIDTH = 12288`).

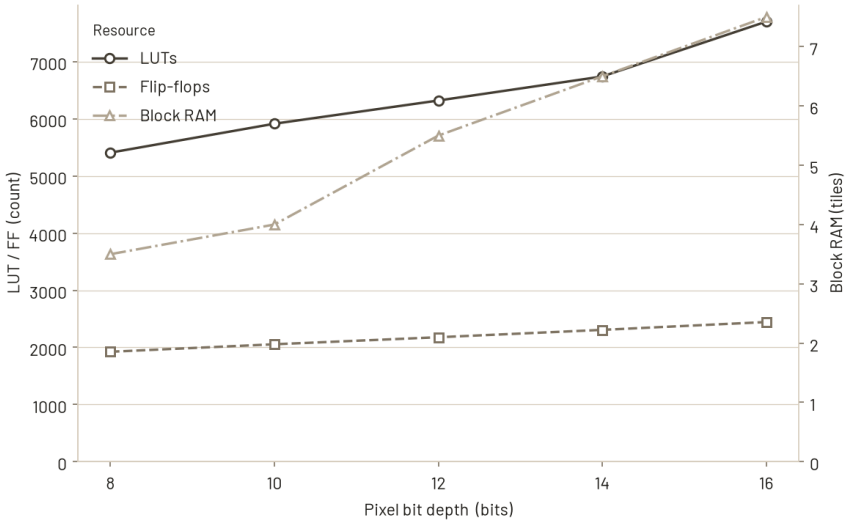


Table 7 Resource usage by pixel bit depth (`MAX_IMAGE_WIDTH = 12288`).

BITNESS	LUT	FF	BRAM
8	5408	1924	3.5
10	5914	2055	4.0
12	6320	2178	5.5
14	6741	2307	6.5
16	7700	2444	7.5

08

Conformance and verification

Verified by simulation with NVC using a layered, self-checking suite run at both RTL and post-synthesis (gate-level) stages:

- **OSVVM** — Per-module tests against independent T.87-derived reference models, top-level control-plane stress (reset injection, backpressure, back-to-back images, dimension fallback), and AXI wrapper tests (byte-exact AXI4-Stream pass-through at 8-, 12- and 16-bit, AXI4-Lite register map, live reconfiguration, mid-image abort), all with requirements tracking.
- **Coverage** — OSVVM functional coverage with NVC structural code coverage.
- **Golden model** — Byte-exact vs CharLS on all 5555 simulation-size images (up to 0.5 MP, 8–16 bit) from reset, 6000 same-size images streamed with no reset between them, plus the ISO/IEC 14495-1 vectors.
- **Design contracts** — Embedded PSL assertions (ready/valid handshake protocol, internal handshakes) on every run.
- **Post-synthesis** — The stress test and a golden-model subset re-run on the synthesized netlist.
- **Hardware-in-the-loop** — The golden-model comparison on silicon: the full 5639-image corpus, including the 84 full-size originals, and the 6000 no-reset stream images, encoded on a PYNQ-Z2 FPGA and byte-compared against CharLS. Run by the EncodeOverEthernet project in OpenJLS-Demos.

The golden corpus (Table 8) combines natural photographs (USC-SIPI, imagecompression.info), which exist only at 8 and 16 bits, with natural-derived crops of the 16-bit photographs rescaled to each depth 9–16, seeded white and $1/f^{\beta}$ noise probes, and boundary/adversarial stress probes. Images over 0.5 MP are sliced into full-width strips for simulation; their 84 full-size originals are part of the corpus but run on hardware only.

Table 8 Golden-model corpus per bit depth.

BITS	NATURAL	NATURAL DERIVED	NOISE	STRESS	FULL-SIZE	TOTAL
8	363	0	300	0	69	732
9	0	300	300	2	0	602
10	0	301	300	2	0	603
11	0	300	300	2	0	602
12	0	300	300	2	0	602
13	0	300	300	2	0	602
14	0	301	300	2	0	603
15	0	300	300	2	0	602
16	45	300	300	31	15	691
All	408	2402	2700	45	84	5639

Table 9 Latest regression snapshot.

SUITE	STATUS	TEST/COV	SUMMARY
NVC code coverage	info	99.8%	Statement coverage[9]
OSVVM suite	PASS	100%	126 tests[10]
Golden model	PASS	100%	11555/11555 images[11]
Post-synth OSVVM	PASS	100%	Control-plane stress[12]
Post-synth golden model	PASS	100%	167/167 images[13]
HIL golden model	PASS	100%	11639/11639 images[14]

09

Revision and contact

Table 10 Revision history.

VERSION	DATE	NOTES
1.0	2026-06	Initial release.
1.1	2026-07	Packaged IPs for Vivado.
1.2	2026-07	Isentropic branding; cores repackaged under the `isentropic` vendor.
1.3	2026-09	Frequency increased ≈15%, LUT reduction ≈16%, BRAM up 1 tile.
1.3.1	2026-10	Source cleanup; verification extended, no functional change.
1.3.2	2026-10	Fix per-image queue overflow on back-to-back images of 8 pixels or fewer.
1.3.3	2026-11	Fix adjacent images merging in the byte stuffer.
1.4	2026-11	Frequency up 18–44%, FF up ≈7%, BRAM up 1–2 tiles at widths from 32k.

License. OpenJLS is dual-licensed: free under GPL v3 for any GPL-compliant use, or under a commercial license for proprietary products without GPL obligations. Evaluation — clone, simulate, synthesize, test — is un-

[9] Per-DUT-file breakdown.
[10] 8 948 681 affirmations (module + top + AXI wrappers).
[11] Byte-exact vs CharLS: 5555 from reset, 6000 streamed with no reset.
[12] 153 946 affirmations on the gate-level netlist.
[13] 8-bit images up to 0.05 MP, byte-exact vs CharLS.
[14] Byte-exact vs CharLS on PYNQ-Z2 silicon at 71.43 MHz: 5639 from reset, 6000 streamed with no reset.

restricted; a commercial license is required only to ship a product.

OpenJLS is developed and maintained by Isentropic. For commercial licensing, technical questions, or collaboration inquiries, contact **contact@isentropic.com.br**.

References

[1] ITU-T T.87 | ISO/IEC 14495-1, *Lossless and near-lossless compression of continuous-tone still images — Baseline (JPEG-LS)*.

[2] Y. M. Mert, *Key Architectural Optimizations for Hardware Efficient JPEG-LS Encoder*, IEEE, 2018.

[3] CharLS, JPEG-LS reference C++ library, `https://github.com/team-charls/charls`.

[4] open-logic, vendor-agnostic VHDL standard library, `https://github.com/open-logic/open-logic`.